

AI Agent Workflow Builder

Step-by-Step Build Guide & Folder Structure

[nhost](#) • [Hasura](#) • [PostgreSQL](#) • [GraphQL](#) • [Next.js](#)

This guide breaks the assignment into an ordered build sequence, with the reasoning for the order, and a complete target folder structure for the repository. The project is not graded piece-by-piece — it is proven with one live end-to-end scenario — so the build order below is chosen so that each step gives you something you can verify before moving to the next, and so that the riskiest parts (permissions, the orchestration Action, live subscriptions) are built on a foundation that is already tested.

How to use this document

- **Part 1** is the ordered build guide — eleven steps, each with what to do, why it's in that position, and what "done" looks like for that step.
- **Part 2** is the complete target folder structure for the repository (backend + frontend).
- **Part 3** is a quick lookup table mapping each requirement in the assignment to the exact file(s) or folder(s) responsible for it, so nothing gets missed.

Part 1 — Ordered Build Guide

Work through these in order. Each step builds on the one before it, and later steps assume earlier ones are already working — don't skip ahead to the orchestration Action before permissions are tested, and don't build the frontend before subscriptions exist to power it.

STEP 1 Set up the backend skeleton

- Install the nhost CLI and run `nhost init` to scaffold a new project; run `nhost up` to start Postgres, Hasura, Auth, Storage and Functions locally.
- Open the Hasura console (URL + admin secret are printed by the CLI) and confirm it loads with an empty schema.
- Create the GitHub repo now, with the nhost project as the root, and commit this empty scaffold as your first commit — migrations and metadata from every later step will diff cleanly against it.

***Why here:** Everything else — schema, permissions, Actions, subscriptions — runs on top of this. If it isn't solid, every later step becomes harder to debug.*

STEP 2 Build the database tables (data model)

- Create tables in dependency order: `organizations` → `org_members` → `workflows` → `workflow_steps` & `workflow_triggers` → `workflow_runs` → `step_runs`.
- Add the usage-quota fields to `organizations` (e.g. `quota_limit`, `quota_used`, `quota_period_start`).
- Add a `status` column to `workflow_runs` that explicitly supports a **paused** value, and an `approved_by` / `approved_at` pair on `step_runs`.
- Track every table in Hasura and wire up the GraphQL relationships (`org`→`members`, `org`→`workflows`, `workflow`→`steps/triggers`, `workflow`→`runs`, `run`→`step_runs`).
- Add one aggregation — a Postgres view or computed field for org-level usage this month or average run duration — and track it in Hasura so it's queryable.

***Why here:** Field names are flexible, but these relationships are exactly what the permission layers and the orchestration Action will query against — get this wrong and you'll be migrating mid-build.*

STEP 3 Lock down permissions — Layer 1 (org + role)

- For every table, add Hasura permission rules per role (owner / editor / viewer) that filter rows through `org_members` using the caller's session user id — role alone is never sufficient, every rule must also scope to the caller's own org.
- Viewers get read-only select permissions; editors get select/insert/update on workflow-building tables but not on org membership; owners get full control including membership.
- Create two test organizations with test users in each role, and manually verify in the Hasura console (or via GraphQL requests with different JWTs) that cross-org reads are blocked before writing a single line of custom logic.

***Why here:** This is the foundation the whole "cross-org isolation" evaluation criterion rests on. It's declarative config, not code, so test it exhaustively now while it's cheap to fix.*

STEP 4 Build the orchestration engine (triggerWorkflowRun Action)

- Define the `triggerWorkflowRun(workflow_id)` Hasura Action and point it at an nhost Function.
- Handler logic, in order: verify caller is owner/editor in the workflow's org → check the org's quota isn't exhausted → create the `workflow_run` row → loop through `workflow_steps` in order, executing each by type.
- `llm_call` and `http_request` steps make real external calls with at least one retry on failure; `db_write` writes into your own tables; `conditional_branch` reads the previous step's output and picks a path.
- Update the matching `step_runs` row (status/input/output/error/attempt count) before and after every step executes, so progress is visible in the database throughout, not just at the end.

***Why here:** This is the core of the assignment and the highest-weighted piece of the eval. Build it as a small, testable state machine rather than one long function — you'll extend it in the next two steps.*

STEP 5 Add the approval-gate pause, and the approveStep Action

- In the step loop, when a `approval_gate` step is reached, set the run's status to **paused** and stop executing — don't move to the next step.
- Define a second Action, `approveStep(step_run_id)`, whose handler checks the approver's role in that org directly in code (this is a mid-execution decision, not a row read/write, so it can't be a Hasura permission alone).
- On approval, record `approved_by/approved_at`, set the run back to running, and resume the step loop from the next step.

***Why here:** The spec calls this out explicitly as needing Action-handler logic, not a database permission — it's one of the clearest signals of what graders will specifically check.*

STEP 6 Add Layer 2 step-level permission gating

- Restrict who can add sensitive step types at creation time: only an owner may add a `db_write` step, a `webhook` trigger, or a `notify` step. This can be a Hasura permission condition on the step's `type/role` column, kept separate from the approval-gate logic in Step 5.
- Re-test with your two organizations and three roles: confirm an editor can build ordinary steps but is rejected when adding a restricted step type, in both orgs independently.

***Why here:** Keeping this as a declarative Hasura rule (vs. more Action code) makes the distinction between the two permission layers visible in your metadata diff — useful for the write-up and for graders.*

STEP 7 Wire up a second trigger type

- Implement at least one of: a Hasura Action functioning as a webhook endpoint external systems can POST to; a Hasura Cron Trigger calling a scheduled nhost Function; or a Hasura Event Trigger that watches a table and starts a run on row change.
- Whichever you choose, it must call the same underlying run-starting logic as `triggerWorkflowRun` so behavior stays consistent regardless of how a run started.
- Implement `notify` itself as a Hasura Event Trigger reacting to `step_runs` changes, per the spec.

***Why here:** The final scenario explicitly requires starting a workflow two different ways — this step is not optional even though the spec lists it after the core Action.*

STEP 8 Add live subscriptions

- Write a GraphQL subscription on `step_runs` filtered by `workflow_run_id`, and confirm it emits an update every time the orchestration Action writes to that table (from Step 4/5).
- Confirm the **paused / awaiting approval** state is a value the subscription surfaces directly, not something the frontend has to infer.

***Why here:** Test this with a GraphQL client (or the Hasura console) before touching the frontend — isolating subscription bugs is much harder once React is in the mix.*

STEP 9 Build the frontend (Next.js)

- Wire up nhost Auth and an org context/switcher.
- Build the workflow builder screen: add/reorder steps of different types, attach a trigger.
- Add a Run button (hidden for viewers), a live per-step status view fed by the Step 8 subscription, and an approve/reject panel that appears when a run is paused.
- Add a small usage/quota indicator reading the aggregation from Step 2.

***Why here:** Build against the already-working backend from Steps 1–8 rather than mocking data — if the backend is solid, the frontend mostly becomes wiring, not new logic.*

STEP 10 Rehearse the final demo scenario end-to-end

- Two orgs, each with real users in each role.
- In Org A, build a workflow with 3+ step types including an `llm_call`, an `http_request`, and a `conditional_branch` that changes behavior based on the LLM's output.
- Start it once manually, once via your Step 7 trigger.
- Let it hit the `approval_gate`, confirm it pauses, approve as an owner/editor in that org.
- Watch the live status stream with no refresh, including the paused state.
- Log in as an Org B user and confirm they cannot see, trigger, or approve anything in Org A — including by directly guessing/pasting an Org A workflow or run id.
- Record this walkthrough.

Why here: This is the entire grading mechanism. If any single piece from Steps 1–9 is weak, it will visibly break here — treat this rehearsal as your real test suite.

STEP 11 Write the deliverables

- README with local setup instructions, including whether the LLM call is real or stubbed and why.
- Deploy the frontend (Vercel or similar) and confirm the hosted URL works standalone.
- Export Hasura metadata and migrations into the repo so schema, relationships, and both permission layers are visible without running the project.
- A ~1 page write-up: schema reasoning, how the two permission layers are enforced differently, and how the approval-gate pause/resume is implemented.
- Attach the recording from Step 10.

Why here: Reviewers are told to open the live app and read exported metadata — not just read source, so these exports and the hosted URL are not optional polish.

Part 2 — Target Folder Structure

A complete repository layout matching the build guide above. The backend lives under `nhost/` (nhost CLI project: migrations, Hasura metadata, serverless Functions), and the frontend lives under `frontend/` (Next.js app).

```
agent-workflow-builder/  
|-- README.md           # setup, run instructions, stub vs real LLM note  
|-- WRITEUP.md          # ~1 page: schema + 2-layer permission reasoning  
|-- .env.example  
`-- .gitignore
```

Backend — `nhost/`

Migrations

```
nhost/migrations/                # Postgres schema, plain SQL, timestamp-ordered
|-- default/
|  |-- 000001_organizations/up.sql
|  |-- 000002_org_members/up.sql
|  |-- 000003_workflows/up.sql
|  |-- 000004_workflow_steps/up.sql
|  |-- 000005_workflow_triggers/up.sql
|  |-- 000006_workflow_runs/up.sql
|  |-- 000007_step_runs/up.sql
|-- 000008_usage_view/up.sql      # aggregation (Part 1, Step 2)
```

Hasura metadata

```
nhost/metadata/                  # Hasura metadata (source of truth, exported)
|-- version.yaml
|-- actions.yaml                 # triggerWorkflowRun / approveStep contracts
|-- actions.graphql
|-- cron_triggers.yaml           # if using cron for the Scheduled trigger type
|-- databases/default/tables/
|  |-- public_organizations.yaml  # perms: Layer 1
|  |-- public_org_members.yaml    # perms: Layer 1
|  |-- public_workflows.yaml      # perms: Layer 1
|  |-- public_workflow_steps.yaml # perms: Layer 1 + Layer 2 (step type)
|  |-- public_workflow_triggers.yaml # perms: Layer 1 + Layer 2 (webhook)
|  |-- public_workflow_runs.yaml  # perms: Layer 1
|-- public_step_runs.yaml         # perms: Layer 1, event trigger: notify
```

Serverless functions

```
nhost/functions/          # nhost Serverless Functions (Node/TS)
|-- trigger-workflow-run.ts # Action handler: triggerWorkflowRun [Step 4]
|-- approve-step.ts        # Action handler: approveStep [Step 5]
|-- inbound-webhook.ts     # Webhook trigger endpoint [Step 7]
|-- scheduled-run.ts       # Cron trigger webhook target [Step 7]
|-- notify-webhook.ts      # Event Trigger target for `notify` [Step 7]
`-- lib/
    |-- steps/
    |   |-- llmCall.ts
    |   |-- httpRequest.ts
    |   |-- dbWrite.ts
    |   |-- notify.ts
    |   |-- conditionalBranch.ts
    |   `-- approvalGate.ts
    |-- permissions.ts      # Layer 2 role checks (approveStep) [Step 5]
    |-- hasuraClient.ts     # admin-secret GraphQL client
    |-- quota.ts            # quota check + increment [Step 4]
    `-- retry.ts            # shared retry-on-failure helper [Step 4]
```

Frontend — frontend/

```
frontend/src/app/         # Next.js app router
|-- layout.tsx
|-- page.tsx              # dashboard / org switcher
|-- login/page.tsx
`-- workflows/
    |-- page.tsx          # workflow list [Step 9]
    |-- [id]/page.tsx     # workflow builder [Step 9]
    `-- [id]/run/[runId]/page.tsx # live run view [Step 9]
```

```
frontend/src/             # components, graphql, lib, context
|-- components/
|   |-- WorkflowBuilder/
|   |   |-- StepList.tsx
|   |   |-- StepEditor.tsx
|   |   `-- TriggerPicker.tsx
|   |-- RunView/
|   |   |-- StepRunTimeline.tsx # subscription consumer [Step 8]
|   |   `-- ApprovalPanel.tsx  # calls approveStep [Step 5]
|   |-- QuotaBadge.tsx
|   `-- RoleGate.tsx          # hides Run for viewers
|-- graphql/
|   |-- queries.ts            # org workflows + steps/triggers + last run
|   |-- mutations.ts          # create/edit workflow, approve
|   `-- subscriptions.ts     # step_runs by workflow_run_id
|-- lib/
|   |-- nhost.ts              # nhost client init
|   `-- apollo.ts             # Apollo client + auth link
`-- context/
    `-- OrgContext.tsx
```


Field and file names above are suggestions, not requirements — the assignment explicitly leaves naming to you. What matters is that the relationships (org → members → workflows → steps/triggers, workflow → runs → step_runs) hold, and that both permission layers and the Action handlers are easy for a reviewer to locate.

Part 3 — Requirement → File Lookup

A quick reference mapping each thing the assignment (and the grading criteria) asks for to where it lives in the folder structure above.

Requirement	Where it lives
Org + role scoping (Layer 1)	nhost/metadata/databases/.../*.yaml (per-table permissions)
Step-level gating (Layer 2, creation)	public_workflow_steps.yaml / public_workflow_triggers.yaml permission conditions
Step-level gating (Layer 2, approval)	nhost/functions/approve-step.ts + lib/permissions.ts
triggerWorkflowRun Action	nhost/metadata/actions.yaml + nhost/functions/trigger-workflow-run.ts
Retry on failure (llm_call / http_request)	nhost/functions/lib/retry.ts, used by lib/steps/llmCall.ts + httpRequest.ts
Quota check + increment	nhost/functions/lib/quota.ts, called from trigger-workflow-run.ts
approval_gate pause / resume	lib/steps/approvalGate.ts (pause) + approve-step.ts (resume)
notify step (Event Trigger)	metadata table yaml event_triggers section + functions/notify-webhook.ts
Webhook / scheduled / DB-event trigger	functions/inbound-webhook.ts or scheduled-run.ts + metadata/cron_triggers.yaml
Usage aggregation view	migrations/.../000008_usage_view/up.sql, tracked as a Hasura table/view
Live step_runs subscription	frontend/src/graphql/subscriptions.ts + components/RunView/StepRunTimeline.tsx
Cross-org isolation proof	Part 1, Step 10 rehearsal – relies on Layer 1 permissions being correct
Quota indicator (frontend)	frontend/src/components/QuotaBadge.tsx
Run button hidden for viewers	frontend/src/components/RoleGate.tsx

Build order recap: skeleton → schema → Layer 1 permissions → orchestration Action → approval gate → Layer 2 permissions → second trigger → subscriptions → frontend → rehearse the scenario → write it up.